Group 3: Anh, Nirvaan, Shital
QAC 387: Final Report
Github Repo
Professor Rose
15 May 2026

Introduction

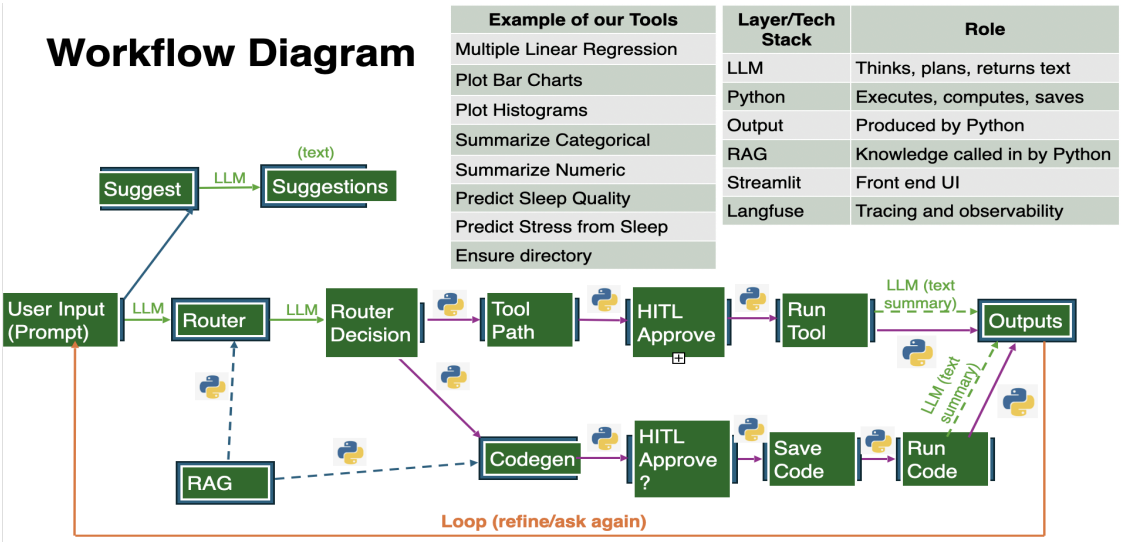
Millions of people use their smartphones every day, and many of them are unaware of how their smartphone usage might be impacting their sleep. Given how important sleep is to overall health, and how ubiquitous smartphones are, the question of how one's smartphone usage might be impacting sleep is an important one. The primary barrier between data and online users is not a lack of data, but rather data literacy. By automating the data to insights process with LLMs, we can help people understand the relationship between their smartphone usage and their sleep. They'll be able to ask questions and get data backed answers in clear and simple English, essentially turning raw data into actionable insights.

Motivation & Background

With so much research happening at Wesleyan and other institutions, we hope our AI agent can be used to help translate scientific knowledge into real world impact. Inspired by the research done at the Sleep and Psychological Adjustment Lab (S.P.A. Lab) at Wesleyan, we wanted to create an agent that can help people without a background in data analysis or sleep research to understand how their phones may be impacting their sleep and therefore their quality of life. We hope that this agent can be used to help build back trust in research that has recently dropped along with funding for science. By allowing people to directly interact with the research they are funding, our aim is that we can improve people's lives and while also help encourage broader understanding of the importance of scientific research. Science communication is essential, and it can feel like the research done at Wesleyan and similar institutions is disconnected from people's lives. Our hope is to show that, with the help of our agent, you do not need a background in research or data analysis to understand research based outcomes.

We used a synthetic dataset (Screen Time, Sleep & Stress Analysis Dataset) sourced from Kaggle for data privacy reasons with 15,000 entries and 13 columns. Our project aims to improve health by reducing the barrier to understand cutting edge research backed insights found at institutions like Wesleyan.

Development Overview



Example of our Tools
Multiple Linear Regression
Plot Bar Charts
Plot Histograms
Summarize Categorical
Summarize Numeric
Predict Sleep Quality
Predict Stress from Sleep
Ensure directory

Layer/Tech Stack	Role
LLM	Thinks, plans, returns text
Python	Executes, computes, saves
Output	Produced by Python
RAG	Knowledge called in by Python
Streamlit	Front end UI
Langfuse	Tracing and observability

The technical architecture of our application is built around a modular agentic workflow that is designed to handle data at a large scale. We utilized a "Router" logic to manage user intent, ensuring that the system could distinguish between general inquiries versus complex statistical tasks. Our primary tech stack includes GPT-4o as the central reasoning large language model that thinks, plans, and returns text, Langchain for managing tool interactions and prompt sequencing, and Streamlit for the front-end user interface. To maintain transparency and accountability, we integrated Langfuse as an observability layer to trace every step of the model's logic (which made it easier for us to traceback to any bugs we found in our code). Our layer also includes Python code which executes, computes, and saves, the Output which is produced by Python code and Retrieval Augmentation Generation (RAG) which is generated by Python.

As illustrated in our workflow diagram, our system functions as a loop. When a user submits a prompt, it is first evaluated by a Router which determines the appropriate path: generating suggestions, providing a qualitative summary, or executing a quantitative tool. This high-level setup allows the application to remain responsive while maintaining the strict logic required for data analysis. By separating the "thinking" aspect of the LLM from the "doing" aspect of the Python execution, we created a robust environment capable of processing our 50,000 row dataset without hitting token limits or compromising on mathematical accuracy.

Development Process

The development process was defined by an iterative approach to prompt engineering and system reliability. We initially focused on a system-prompting that utilized few shot prompting, providing the model with specific examples of how to map conversational language to our

dataset's 13 variables. This step was critical for ensuring that the model recognized that a term like "caffeine use" referred specifically to the Caffein_Intake_Cups column. To support this, we implemented a schema-driven RAG strategy. Instead of retrieving raw data chunks, we provided the LLM with dataset's metadata, which is the column names, data types, and ranges. This step allowed the model to maintain a high-level "map" of the data, which it used to generate precise, error-free queries.

A major turning point in our process was the pivot from a standard "Chat-with-CSV" agent to a structured Code-generation Pipeline. Our early testing, with the help of Langfuse tracing revealed that allowing the LLM to interpret the data directly from the dataset often led to hallucinations or "fuzzy" math. In order to resolve this issue we re-engineered the process so that the LLM would actually write Python code instead of an answer. That code was then executed in a local REPL sandbox against our dataframe, with the results being passed back to the LLM for a final summary. This iteration ensured nearly 100% statistical accuracy. Throughout these phases, we continuously used Langfuse tracing to monitor the logic at each step, allowing us to see exactly where the router or code-generator needed refinement. This iterative loop was essential in transforming a general AI model into a specialized behavioral analysis tool.

Testing & Validation

From our test attempts, the core setup and environment always worked smoothly. RAG also loaded with 117 chunks from the knowledge base of analysis guides and tools documentation. Langfuse was operating as expected which allowed us to look at router decisions and retrieval steps, which helped debug failed tests and compare successful ones. With the Streamlit user interface, the schema, column details and tools list were all clear. Our agent's output was aimed to be on the more concise and general side which enables an individual not familiar with data analysis or research to use it effectively.

Our best performance would be from code generation which was robust to the extent that vague prompts like "help me analyze this dataset" produced complete Python workflows with proper error handling. For guardrails we had human in the loop to oversee reviewing code before saving and running it. Upon receiving out of scope questions, the agent declined requests which further confirm that guardrails did work as needed. Some of the tool execution didn't always work well, specifically while summarize and multiple linear regression were fine, but despite bar charts existing as a tool we found that our router defaulted to codegen.

The takeaway from our testing and validations is that our agent is decisive about its limitations and demonstrates its reasoning well but human oversight is still very necessary.

Challenges & Limitations

Challenges we ran into included path management, routing, and hallucinations. In terms of routing our agent sometimes chooses the wrong tools, like outputting a plot when asked to run a regression. Additionally, the agent also sometimes misinterprets questions like creating a bar graph by the wrong variable. Moving onto the router, it is also not fully consistent. Sometimes when asked a question that is unrelated to the dataset, instead of rejecting it, the agent will output a summary of the dataset. Path management was also a challenge as our tools point towards different folders in our reports directory. Therefore, when the agent runs a tool it occasionally is unable to find the correct directory leading to it failing. Finally, the agent sometimes, but rarely, hallucinates variables that do not exist.

Next Steps

Next steps include firstly improving our tools. Currently there are many different parameters used for pathways that the tools take, and this can cause issues where the tool cannot fully run because of path issues (sometimes points towards a directory that doesn't exist). Next steps to fix this issue would be tweaking our code to have one reports folder where everything can go. This is not an issue with Streamlit as outputs can be seen in it directly under the output tab, but we would like our back end architecture to mirror that.

Additionally, we want to improve routing. Future steps could include adding a confidence score to routing, and if the score is below a certain threshold, it can fall back to codegen. We can also add more examples to the router to improve accuracy in what it chooses. When codegen is chosen, we want to ensure that columns exist before codegen to protect against hallucinations, adding safeguards to our instructions can help this. Our router often chooses codegen even over tools, so we want to be careful with how we implement this.

Finally, next steps to refine our agent and experience can happen through adding a dashboard and working on being able to apply our agent to other datasets. For the dashboard, users should be able to input some information like sleep, gender, age, and occupation to receive a predicted sleep time and caffeine intake recommendations based off of the dataset that the agent analysed. Being able to apply the agent to other datasets to answer questions would help bridge scientific research to people's lives and the decisions they make. For example, research in the Cognitive Development Lab can be leveraged to help parents understand their child’s number learning and language development.

Summary

Our agent is able to translate research findings into easily understandable insights for anyone to understand. Helping bridge one of the key issues in science, getting people excited, helping them understand, and being able to apply cutting edge scientific knowledge to their lives. We hope our agent can help rebuild trust in science while also improving public understanding of scientific research.

Throughout this process we have learned to appreciate the iterative developmental process that this project has taken throughout the semester. In terms of AI safety the importance of humans in the loop (HITL) can not be overstated. By needing to approve code before running we can catch hallucinated variables before running and users can reject running that code. Moreover, it would help guard against agents pushing dangerous code. RAG also has been something interesting to experiment with, and seems to reduce the hallucination of non-existent variables. Throughout this process Langfuse has been especially helpful in identifying where failures occur especially with our router, and helps us know when our model is being too slow. Some take always is that paths should always be absolute, not relative, to avoid running into problems. Furthermore, tool descriptions matter more than we expected, by having better descriptions we can greatly improve router accuracy as well as ensure our tools are working as expected.